Reinforcement Learning Lab

Lesson 1: Multi-Armed Bandit

Alberto Castellini, Riccardo Fidanza

University of Verona
email: alberto.castellini@univr.it, riccardo.fidanza@studenti.univr.it

Academic Year 2025-26

Initial Setup Process - 1

The first step for the setup of the laboratory environment is to install miniconda, a light version of the Conda package manager.

- Download the *Miniconda* package manager: `https://docs.conda.io/en/latest/miniconda.html`
- Install *Miniconda*
  - On *Linux* run *Miniconda3-latest-Linux-xx.sh*, remember to provide the executions permissions to the file.
  - On *Windows*, double-click the installer; in the installation phase, ensure to install *Anaconda Prompt* and use it for the next steps.

Virtual Environment

For python virtual environments' users (*venv*), the Miniconda installation can be avoided.

Initial Setup Process - 2

The second step is downloading the official laboratory repository and creating the working environment. The following commands must be run in the *Linux Shell* or in the *Anaconda Prompt* on Windows.

- Clone the official Lab repository:
  git clone https://github.com/Isla-lab/RL-lab
- Create the environment and install the required packages:
  conda env create -f RL-Lab/tools/rl-lab-environment.yml
- Before running the scripts, remember to activate the environment:
  conda activate rl-lab
- The first time you activate the environment install gym: *pip3 install gym*

Git Installation

The installation of Git may be required. For Linux users, just run *sudo apt-get install git*. For Windows users, refer to the official page `https://git-scm.com/downloads`.

Assignments Structure

In general, the requirement for a laboratory lesson is to complete the partial python code provided, for example:

```
RL-Lab/lessons/lesson_n_code.py
```

The **main** function is always provided in a Python script. Once the code is completed, students have to run the script to obtain all the results in the output on the console. The expected results can be found in the *.txt* file corresponding to the lesson, for example:

```
RL-Lab/results/lesson_n_results.txt
```

Disclaimer

Given the stochasticity of the algorithms that we will run in the course, the results can be slightly different from the ones reported in the *.txt* file. The relevant point is the general trend.

Today Assignment

In today's lesson, we will implement the Multi-Armed Bandit Environment and the Simple Bandit Algorithm algorithm to solve it. In particular, the file to complete is:

```
RL-Lab/lessons/lesson_1_code.py
```

Inside the file, a python class and a function are partially implemented. The objective of this lesson is to complete it.

- **class MultiArmedBandit()**
- **def banditAlgorithm()**

Expected results can be found in:

```
RL-Lab/results/lesson_1_results.txt
```

Environment: Multi-Armed Testbed

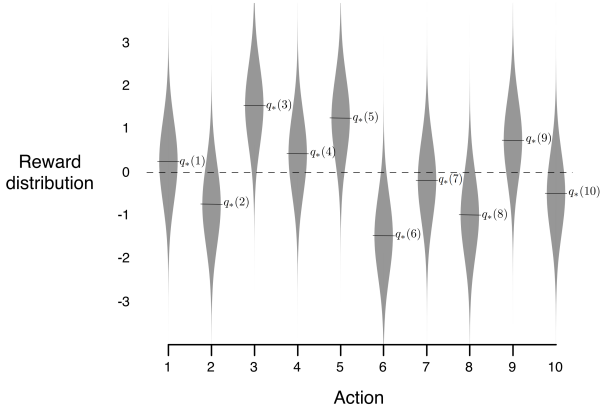


Figure: Visual explanation of the Multi-Armed Testbed environment, from the Sutton and Barto book *Reinforcement Learning: An Introduction*

- The *Multi-Armed Testbed* environment consists of a set of N possible actions, from 1 to N. A mean value ($q^*(a)$) has been assigned to each action, sampled from a normal distribution with $\mu = 0$ and $\sigma^2 = 1$.
- For a given action a, the environment should return a reward sampled from a normal distribution with $\mu = q^*(a)$ and $\sigma^2 = 1$.

Algorithm: Simple Bandit

A simple bandit algorithm

Initialize, for $a = 1$ to k:

$Q(a) \leftarrow 0$

$N(a) \leftarrow 0$

Loop forever:

$$A \leftarrow \begin{cases} \arg\max_a Q(a) & \text{with probability } 1-\varepsilon \quad \text{(breaking ties randomly)} \\ \text{a random action} & \text{with probability } \varepsilon \end{cases}$$

$R \leftarrow bandit(A)$

$N(A) \leftarrow N(A) + 1$

$Q(A) \leftarrow Q(A) + \frac{1}{N(A)}\big[R - Q(A)\big]$

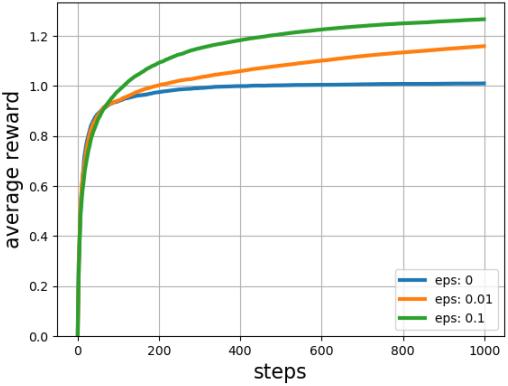
Figure: Pseudocode for Simple Bandit Algorithm, from the Sutton and Barto book *Reinforcement Learning: An Introduction*

Simple Bandit Algorithm applied to 10-Armed Testbed

The suggested solution exploits a NumPy function to sample from a normal distribution, numpy.random.normal(). More details can be found on the official website (here).

Seeding

Given the (particularly) high stochasticity of the method and the environment, for this lesson, we fixed a random seed equal to 6.

Hint (Expected results)

The plot on the right is the expected result. Notice that the best results have been obtained with eps=0.1, while the worst one with eps=0 (i.e., no exploration).